

# MODULE 2.2: UNSUPERVISED LEARNING

## Contents

|                                                                  |   |
|------------------------------------------------------------------|---|
| Matrix Factorization.....                                        | 1 |
| Latent Factors and Prediction.....                               | 2 |
| Optimization .....                                               | 2 |
| Generative Models: Mixture Models and Latent Factor Models ..... | 3 |
| 1. Mixture Models.....                                           | 3 |
| 2. Latent Factor Models.....                                     | 4 |
| Gaussian Mixture Models (GMM).....                               | 5 |
| Parameters in GMM .....                                          | 5 |
| How GMM Works in Practice (Summary) .....                        | 6 |
| Applications of GMM .....                                        | 6 |
| Expectation-Maximization (EM) algorithm .....                    | 6 |
| Need for EM in Latent Variable Models .....                      | 6 |
| EM algorithm in detail.....                                      | 7 |
| Use in Training Gaussian Mixture Models (GMM) .....              | 8 |

## Matrix Factorization

- Matrix Factorization is a technique used mainly in **recommender systems** (like Netflix, Amazon, Spotify) to predict missing user-item ratings by breaking a **large sparse matrix** into two **lower-dimensional** matrices.
- **Matrix Factorization (MF)** is a core technique in **Collaborative Filtering (CF)** for Recommender Systems. It is a model-based approach that addresses the data sparsity problem by projecting users and items into a shared, low-dimensional space of **latent factors** (or embeddings).

Matrix Factorization works by taking the sparse **User-Item Interaction Matrix (R)** and approximating it as the product of two much smaller, dense matrices: a **User Factor Matrix (U)** and an **Item Factor Matrix (V)**.

$$R_{m \times n} \approx U_{m \times k} \times V_{k \times n}^T$$

- **R (Interaction Matrix):** The input matrix of size m users by n items, containing known ratings or implicit interactions (e.g., clicks, views). Most entries are empty (sparse).
- **U (User Factor Matrix):** A matrix where each row  $U_u$  is a **latent vector (embedding)** representing user u.

- **V (Item Factor Matrix):** A matrix where each column  $V_i$  is a **latent vector (embedding)** representing item  $i$ .
- **k (Dimensionality):** The size of the latent space (typically small, e.g., 10 to 100). This dimension  $k$  represents the number of hidden features (latent factors) that govern user preferences and item characteristics.

## Latent Factors and Prediction

The model learns the vectors in  $U$  and  $V$  by trying to minimize the difference between the observed ratings in  $R$  and the reconstructed ratings predicted by  $U \cdot V^T$ .

### a. Latent Factors

Each dimension in the  $k$ -dimensional latent space corresponds to an unobserved, abstract feature.

- **User Embeddings ( $U_u$ ):** The vector for user  $U$  represents their **preference strength** for those  $k$  factors. For example, a high value in the first dimension might mean the user strongly likes "action" movies.
- **Item Embeddings ( $V_i$ ):** The vector for item  $i$  represents the **degree to which the item exhibits** those  $k$  factors. A high value in the first dimension might mean the item is heavily "action-oriented."

### b. Making a Prediction

The predicted rating ( $\hat{r}_{u,i}$ ) for user  $U$  on item  $i$  is calculated as the **dot product** of their respective latent vectors:

$$\hat{r}_{u,i} = U_u \cdot V_i^T$$

The dot product is high if the user's preference factors align strongly with the item's characteristic factors.

## Optimization

The core challenge is finding the optimal values for the latent factor matrices  $U$  and  $V$ .

- **Cost Function:** The objective function minimizes the error between the true known ratings and the predicted ratings, often including **L2 regularization** (to prevent overfitting to the sparse training data):

$$\text{Minimize} \quad \sum_{(u,i) \in \text{Observed}} (r_{u,i} - U_u V_i^T)^2 + \lambda(||U||^2 + ||V||^2)$$

- **Optimization Algorithm:** The two most common techniques used to solve this minimization problem are:
  - **Stochastic Gradient Descent (SGD):** Iteratively adjusts the values in  $U_u$  and  $V_i$  for one observed rating at a time to reduce the error.
  - **Alternating Least Squares (ALS):** Alternates between fixing  $U$  and solving for  $V$  (which becomes a simple linear least squares problem) and then fixing  $V$  and solving for  $U$ . This is highly **parallelizable** and preferred for very large datasets.

Q.) 4 Users ( $U_1 - U_4$ ) & 5 Items/Movies ( $M_1 - M_5$ ). Let's Use  $k=2$  (latent factors)  $F_1 = \text{comedy}$ ,  $F_2 = \text{horror}$ .  
 Given, Sparse Interaction matrix ( $R$ ), User Factor matrix ( $U$ ),  
 Item Factor matrix ( $V$ ).

| $R / \begin{matrix} \text{User} \\ \text{Movie} \end{matrix}$ | $M_1$ | $M_2$ | $M_3$ | $M_4$ | $M_5$ |
|---------------------------------------------------------------|-------|-------|-------|-------|-------|
| $U_1$                                                         | 5     |       | 4     |       | 1     |
| $U_2$                                                         | 4     | 2     | 3     | 5     |       |
| $U_3$                                                         | 1     |       |       | 4     | 5     |
| $U_4$                                                         |       | 5     | 1     | 2     |       |

| $U / \begin{matrix} \text{User} \\ \text{Factor} \end{matrix}$ | $F_1$ | $F_2$ |
|----------------------------------------------------------------|-------|-------|
| $U_1$                                                          | 1.8   | -0.2  |
| $U_2$                                                          | 1     | 0.5   |
| $U_3$                                                          | -1.5  | 1.2   |
| $U_4$                                                          | -0.5  | 1.9   |

| $V / \begin{matrix} \text{Movie} \\ \text{Factor} \end{matrix}$ | $F_1$ | $F_2$ |
|-----------------------------------------------------------------|-------|-------|
| $M_1$                                                           | 2.5   | 0.1   |
| $M_2$                                                           | -1    | 2     |
| $M_3$                                                           | 1.2   | -0.5  |
| $M_4$                                                           | 0     | 1     |
| $M_5$                                                           | -1.9  | 0.9   |

$$\text{Ans) } r_{U_1, M_2} = U_1 V_2^T = [1.8 \quad -0.2] \begin{bmatrix} -1 \\ 2 \end{bmatrix} = 1.8(-1) + (-0.2)2 = -1.8 - 0.4 = -2.2$$

$$r_{U_1, M_4} = U_1 V_4^T = [1.8 \quad -0.2] \begin{bmatrix} 0 \\ 1 \end{bmatrix} = -0.2$$

$$r_{U_2, M_5} = [1 \quad 0.5] \begin{bmatrix} -1.9 \\ 0.9 \end{bmatrix} = -1.45$$

$$r_{U_3, M_2} = [-1.5 \quad 1.2] \begin{bmatrix} -1 \\ 2 \end{bmatrix} = 3.9$$

Like this we can find all.

## Generative Models: Mixture Models and Latent Factor Models

- **Generative Models** are a class of probabilistic machine learning models that aim to learn the **underlying joint probability distribution**  $P(X, Z)$  of the observed input data ( $X$ ) and Latent variable ( $Z$ ). Once this distribution is learned, the model can be used to **generate new, synthetic data samples** ( $X_{\text{new}}$ ) that resemble the original training data.
- Mixture Models and Latent Factor Models are two major categories of generative models, both relying on the concept of **latent variables (Z)** to explain the observed data.

### 1. Mixture Models

Mixture models assume that the observed data is generated from a combination (a weighted sum) of several simpler, known probability distributions (components).

- **Mechanism:** The model assumes that each data point belongs to one of  $K$  different **subpopulations** or **components**. The latent variable  $\mathbf{Z}$  is a **discrete variable** representing the specific component (cluster) that generated the data point.
- **Example:**
  - **Gaussian Mixture Model (GMM):** GMM is the canonical example of a mixture model, assuming data comes from a mixture of discrete Gaussian components.
  - **Expectation-Maximization (EM) Algorithm:** is the iterative **algorithm** used to find the maximum likelihood parameters for models with unobserved (latent) variables, including GMMs.

## 2. Latent Factor Models

Latent factor models assume that the high-dimensional observed data is generated from a small set of continuous, unobserved factors that govern the variation in the data.

- **Mechanism:** The model assumes that the observed features ( $\mathbf{X}$ ) are caused by a set of continuous, low-dimensional **latent factors** ( $\mathbf{Z}$ ). The latent variable  $\mathbf{Z}$  is usually a **continuous vector** (an embedding) that captures a compressed representation of the data.
- **Example**
  - **Variational Autoencoders (VAEs):** A modern deep learning implementation of latent factor models.
  - **PCA:** for dimensionality reduction
  - **Matrix Factorization:** This is a latent factor generative model for ratings.

| Feature                          | Mixture Models                                                    | Latent Factor Models                               |
|----------------------------------|-------------------------------------------------------------------|----------------------------------------------------|
| Generative Assumption            | Data comes from a <b>weighted mixture</b> of simple distributions | Data comes from <b>continuous latent factors</b>   |
| Latent Variable ( $\mathbf{Z}$ ) | <b>Discrete</b> (component index)                                 | <b>Continuous</b> (low-dimensional vector)         |
| How Data is Generated            | Each point from <b>one component</b>                              | Each point influenced by <b>multiple factors</b>   |
| Representation                   | Produces <b>clusters</b>                                          | Produces <b>compressed embeddings</b>              |
| Data Structure Captured          | <b>Group/subpopulation structure</b>                              | <b>Hidden feature structure</b>                    |
| Examples                         | GMM                                                               | PCA, Matrix Factorization, VAE                     |
| Key Algorithm / Use              | EM for parameter estimation                                       | Dimensionality reduction + representation learning |

# Gaussian Mixture Models (GMM)

A **Gaussian Mixture Model (GMM)** is a **probabilistic model** that assumes that the data is generated from a mixture of **several Gaussian distributions** (also called components), each with its own parameters.

Each component in the mixture represents a **sub-population** or **cluster**, and the overall data distribution is a **weighted sum** of these individual Gaussians.

## Use in Clustering:

- GMM is used in **unsupervised learning** for clustering data based on the assumption that each cluster is generated by a different Gaussian distribution.
- Unlike k-means, GMM provides **soft clustering**, meaning it gives a **probability** of each point belonging to each cluster.
- It is more **flexible** than k-means since it allows **elliptical clusters** (via covariance), not just spherical ones.

## Parameters in GMM

Each component Gaussian  $k$  has three main parameters:

### a) Mean vector $\mu_k$

- Center of the  $k^{\text{th}}$  Gaussian component.
- Controls the **location** of the component in feature space.

### b) Covariance matrix $\Sigma_k$

- Defines the **shape, orientation, and spread** of the Gaussian.
- In 1D, it is just the variance. In 2D or higher, it can model ellipses or general shapes.
- Determines whether the component is **spherical, elliptical, or rotated**.

### c) Mixing coefficient $\pi_k$

- Represents the **prior probability** of choosing component  $k$ .
- Must satisfy  $\sum \pi_k = 1$  ( $k=1$  to  $K$ )
- Indicates the **relative size or weight** of each Gaussian in the mixture.

Probability Density Function of GMM.

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x; \mu_k, \Sigma_k)$$

Here,

$$\sum_{k=1}^K \pi_k = 1, \pi_k \geq 0$$

$\rightarrow \mathcal{N}(\mu_k, \Sigma_k)$  is multivariate Gaussian pdf.

$$\mathcal{N}(\mu_k, \Sigma_k) = \frac{1}{\sqrt{(2\pi)^d |\Sigma_k|}} e^{-\frac{1}{2}(x-\mu)^T \Sigma^{-1}(x-\mu)}$$

①  $\mu_k$  = mean vector  
②  $\Sigma_k$  = covariance matrix  
③  $\pi_k$  = mixing coefficient (weight of component  $k$ )

## How GMM Works in Practice (Summary)

1. A data point  $x$  is assumed to be generated by first picking a component  $k$  with probability  $\pi_k$ , then sampling  $x$  from  $N(\mu_k, \Sigma_k)$
2. The overall distribution is a combination of these individual Gaussians.
3. Algorithms like **Expectation-Maximization (EM)** are used to estimate the parameters,  $\mu_k$ ,  $\Sigma_k$ ,  $\pi_k$  from the data.

## Applications of GMM

1. **Clustering (Unsupervised Learning):** Provides **soft clustering** and models flexible **elliptical cluster shapes**, making it superior to K-Means for complex data distributions.
2. **Speech Recognition:** Models the **probability distribution of acoustic features** (like MFCCs) for distinct sound units, historically vital for HMM-GMM sequence modeling.
3. **Image Segmentation:** Treats pixel data as Gaussian components to **probabilistically separate objects** from the background based on color or intensity distributions.
4. **Anomaly Detection:** Fits the model to "normal" data; **samples with extremely low likelihood** under the GMM are flagged as anomalies.
5. **Background Subtraction in Videos:** Models the temporal pixel history of a fixed location as a mixture of Gaussians to **isolate moving foreground objects** from the static background.
6. **Density Estimation:** Serves as a flexible, continuous estimator for the **underlying probability distribution** of complex, multimodal data.
7. **Bioinformatics:** Used to cluster **gene expression profiles** and model **population genetics** by identifying hidden subtypes in biomedical datasets.

## Expectation-Maximization (EM) algorithm

The **Expectation-Maximization (EM)** algorithm is an **iterative optimization technique** used to find **maximum likelihood estimates** of parameters in statistical models, particularly when the model involves **latent (hidden or unobserved) variables**.

It alternates between:

- **E-step:** Estimate the hidden variables using current parameters.
- **M-step:** Update parameters to maximize the likelihood based on those estimates.

## Need for EM in Latent Variable Models

### What is a Latent Variable Model?

- A model where some variables (**latent variables**) are not directly observed in the data but are assumed to exist.
- In **GMM**, the latent variable is the **cluster/component assignment** (i.e., which Gaussian each data point came from).

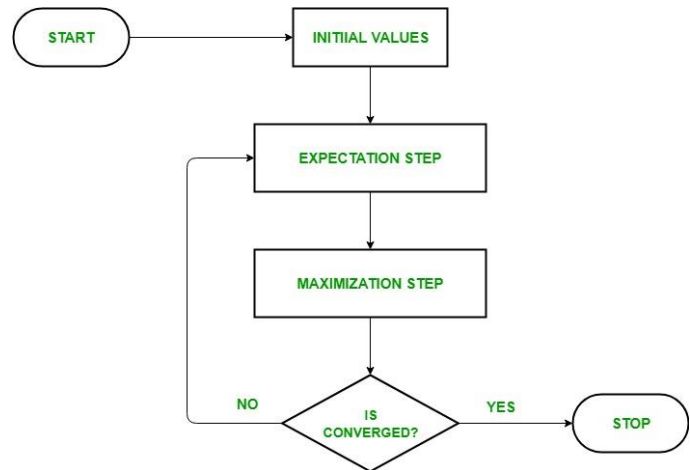
### Why EM is Needed

- The **log-likelihood** of such models becomes complex because we need to marginalize over the hidden variables.

- **Direct maximization** of the log-likelihood becomes **intractable** due to summation inside the log function.
- EM provides a **tractable iterative solution** to find the **maximum likelihood estimates** of parameters.

### EM algorithm in detail.

1. Initially, a set of initial values of the parameters are considered. A set of incomplete observed data is given to the system with the assumption that the observed data comes from a specific model.
2. Expectation step (E-step): Using the observed available data of the dataset, estimate (guess) the values of the missing data. It is basically used to update the variables.
3. Maximization step (M-step): Complete data generated after the expectation (E) step is used in order to update the parameters. It is basically used to update the hypothesis.
4. Repeat step 2 and step 3 until convergence.



### Steps in EM Algorithm

Assume

- $X = \{x_1, x_2, \dots, x_n\}$  be observed data
- $Z =$  latent variables
- $\theta$ : parameters of model (mean, covariance, mixing coefficients)

### Step 1: Estep

Computing posterior probability that a datapoint  $x_i$  was generated by component  $k$ ,

$$P(x \in k) = \frac{\pi_k \mathcal{N}(x | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x | \mu_j, \Sigma_j)}$$

### Step 2: M step

we update the parameters to maximize the expected log likelihood

(i) Mean 
$$\mu_k^{(t+1)} = \frac{\sum_{i=1}^n P(x \in k) \cdot x_i}{\sum_{i=1}^n P(x \in k)}$$

(ii) Covariance 
$$\Sigma_k^{(t+1)} = \frac{\sum_{i=1}^n P(x \in k) (x_i - \mu_k) (x_i - \mu_k)^T}{\sum_{i=1}^n P(x \in k)}$$

(iii) Mixing Coefficient 
$$\pi_k^{(t+1)} = \frac{1}{n} \sum_{i=1}^n P(x \in k)$$

Step 3: Repeat step 1 & 2 until parameters converge or change becomes minimal

## Use in Training Gaussian Mixture Models (GMM)

In GMM:

- We do **not know** which Gaussian component each data point came from  $\rightarrow$  the **latent variable**.
- EM estimates:
  - **Responsibilities** (E-step): Probabilities of assignment to Gaussians
  - **Parameters** (M-step): Means, covariances, and mixing weights of the Gaussians

EM allows us to:

- Fit a GMM to data **even without knowing component labels**
- Perform **soft clustering**
- Estimate full parameters of the mixture model from **unlabelled data**

### Uses of EM Algorithm

1. **Gaussian Mixture Models (GMM):** EM is the standard method to estimate parameters in GMM. Handles unknown cluster assignments (latent variables).
2. **Missing Data Problems:** EM estimates parameters when parts of the data are missing or unobserved.
3. **Hidden Markov Models (HMM):** The Baum-Welch algorithm, used to train HMMs, is a type of EM algorithm.

4. **Image Processing & Segmentation:** EM is used to separate image regions probabilistically based on pixel values.
5. **Medical Diagnosis & Bioinformatics:** Used in probabilistic inference when some patient attributes or biological factors are hidden.

### Advantages of EM Algorithm

1. **Handles Incomplete Data:** Works well when some variables are unobserved or latent.
2. **Simple to Implement:** Each step (E and M) usually has a straightforward closed-form or numerical solution.
3. **Guaranteed Convergence:** EM guarantees that the likelihood does not decrease with each iteration.
4. **Soft Assignment:** Unlike hard clustering (like k-means), EM gives probabilistic (soft) assignments.
5. **Flexible Models:** EM can be applied to many models beyond GMMs (e.g., HMMs, factor analysis, etc.).

### Disadvantages of EM Algorithm

1. **Converges to Local Maxima:** EM can get stuck in local maxima of the likelihood function; results depend on initialization.
2. **Slow Convergence:** May require many iterations to converge, especially near the maximum.
3. **No Guarantee of Global Optimum:** It does not necessarily find the best global parameter estimates.
4. **Sensitive to Initialization:** Poor initial parameter values may lead to poor final results.
5. **Assumes Model Correctness:** Works best when the data truly comes from the assumed model (e.g., mixture of Gaussians). If not, performance degrades.